Parcial practico La última transmisión

Integrantes:

Rodrigo Andres Facio Lince Beltran

Código: T00069253

Luis Gustavo Ruiz Teran

Código: T00067451

Docente:

Aisner José Marrugo Juliao

Fecha:

14 de octubre de 2026

UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

INGENIERIA DE SOFTWARE

2026

1. Introducción y Alcance (Unidad 1 - Fundamentos)

El proyecto consiste en un motor de ficción interactiva por consola desarrollado en Python, guiado por un diseño orientado a datos (*Data-Driven*) para separar la lógica de negocio de los archivos de contenido en JSON.

- Actores principales: El escritor (creador de las historias en JSON) y el lector/jugador (usuario que ejecuta las decisiones por consola).
- Alcance: Incluye la carga estructurada, el filtrado de opciones por condiciones booleanas, la aplicación de efectos, la detección de finales y un validador preventivo de errores. Quedan fuera las interfaces gráficas complejas y la persistencia en bases de datos externas, operando en memoria local.
- Riesgos y mitigación: Se mitiga la rotura de código ante cambios imprevistos mediante arquitectura por capas, y los errores de contenido en los JSON mediante validación estricta a la entrada.

2. Proceso y Metodología Ágil (Unidades 2 y 3 - Procesos y Agilidad)

- Proceso Seleccionado: Desarrollo Incremental y Ágil (basado en Scrum).
- Alternativa Descartada: Cascada (Waterfall), debido a que no permite integrar fácilmente el cambio sorpresa esperado a las 6 horas.
- Prácticas Ágiles Aplicadas:
    1. *Backlog Priorizado:* Ordenar tareas por valor (motor base primero, GUI elaboradas descartadas).
    2. *Test-Driven Development (TDD) Parcial:* Escribir los escenarios de validación antes de consolidar el motor.
- Práctica Descartada: Programación en Pares síncrona (Pair Programming), debido al tiempo limitado de los integrantes y a la sincronización necesaria que no se puede permitir en el momento.
- Definición de Terminado (DoD): Un requisito está terminado cuando su código está integrado, pasa sus pruebas unitarias, carece de "code smells" evidentes, y su funcionamiento es demostrable.

- Riesgo 1: Rotura del motor principal al recibir la solicitud de cambio en la hora 6. *Mitigación:* Estricto encapsulamiento (arquitectura por capas) y cobertura de pruebas de regresión.
- Riesgo 2: Archivos de historia mal formados en producción. *Mitigación:* Validador estricto a la entrada antes de instanciar la clase de estado.
- Responsabilidad Profesional (Datos): El validador garantiza que las fallas del contenido de un escritor no generen un "crash" en el equipo del lector (Safety/Reliability).
- Aprendizaje y Acción de Mejora: (Inicial) Observamos que depurar validaciones con historias completas es lento; *Acción:* Usar JSONs diminutos e inyectados en los tests unitarios en lugar de depender de los archivos de texto completos.

3. Especificación de Requisitos y Decisiones

3.1. Requisitos Funcionales (RF)

- RF-01 (Carga Estructurada - N01): El sistema lee un archivo JSON para inicializar la partida según los campos inicio y variables. *Criterio:* Carga correcta y coincidencia exacta en memoria.
- RF-02 (Filtrado de Opciones - N02): Evalúa el mapa de condiciones de cada opción frente al estado actual, mostrando únicamente las habilitadas. *Criterio:* Las opciones con restricciones no cumplidas se omiten.
- RF-03 (Transición y Efectos - N03): Al seleccionar una opción válida, actualiza las variables de estado y avanza a la escena de destino. *Criterio:* El estado refleja el cambio y se cambia de escena.
- RF-04 (Persistencia en Memoria - N04): Conserva las variables booleanas modificadas durante toda la secuencia de turnos consecutivos.
- RF-05 (Final y Reinicio - N05): Si la escena es final (final: true), omite la selección y permite reiniciar la partida restaurando estado y escena inicial.
- RF-06 (Validación Preventiva - N06): Rechaza antes de iniciar cualquier archivo con errores estructurales (destinos inexistentes, IDs duplicados o campos faltantes) con un error controlado.

3.2. Requisitos No Funcionales (RNF)

- RNF-01 (Modificabilidad): Permite ejecutar historias nuevas cambiando únicamente la ruta del archivo JSON, sin modificar código fuente.
- RNF-02 (Confiabilidad): Ignora entradas de texto inválidas u opciones inhabilitadas sin alterar el estado ni detener la ejecución de la consola.
- RNF-03 (Claridad de Errores): Emite mensajes exactos y detallados ante fallas tipificadas en el contenido de los archivos de entrada.

3.3. Supuestos Clave y Defecto Corregido

- Supuesto inicial y actualización: Inicialmente se asumió un flujo unidireccional; no obstante, a las 6 horas el cliente revocó este acuerdo e incorporó la regla de retroceso de un solo nivel (RF-07).permite efectos múltiples por opción, y considera habilitadas por defecto las condiciones vacías.
- Defecto Detectado (*Softlock*): Si una escena no final se quedaba sin opciones válidas, la consola entraba en bucle. *Corrección:* Se adaptó cli.py para detectar len(opciones) == 0, informando el bloqueo y forzando un reinicio seguro.

4. Modelado y Arquitectura (Unidades 5 y 6)

4.1 Modelos del Software

Para documentar formalmente el comportamiento y la estructura del sistema, se diseñaron cinco modelos principales que se encuentran detallados de forma gráfica en el Anexo PDF:

- Diagrama de Contexto: Muestra los límites del sistema frente a los actores externos (Escritor y Lector) y los archivos de intercambio (.json).
- Diagrama de Casos de Uso: Ilustra las interacciones permitidas al usuario, tales como cargar historia, listar opciones habilitadas, seleccionar camino, registrar efectos de estado y reiniciar la partida.
- Diagrama de Clases del Dominio: Define las entidades principales (Partida, ValidadorHistoria, GestorJSON) y sus relaciones lógicas basadas en programación orientada a objetos en Python.
- Diagrama de Secuencia: Detalla el flujo temporal de mensajes entre la interfaz (cli.py), el motor de reglas (motor.py) y el almacenamiento en memoria durante la selección de una opción con efectos.
- Diagrama de Estados: Representa el ciclo de vida de la partida, transitando entre la carga inicial, la espera de entrada en escena activa, la evaluación de condiciones y el estado terminal de reinicio o finalización.

4.2 Decisiones de Arquitectura

Separación en Capas: El sistema se estructuró de manera desacoplada en tres capas independientes:

1. *Capa de Presentación / CLI (cli.py):* Maneja la interacción por consola, la captura de entradas del usuario y el despliegue formateado del texto narrativo.
2. *Capa de Lógica de Negocio / Motor (motor.py):* Aloja las reglas de validación estructural, la evaluación de condiciones booleanas y la persistencia temporal de variables en memoria.
3. *Capa de Datos (.json):* Archivos externos independientes que contienen la estructura narrativa.

Justificación del Diseño Orientado a Datos: Se adoptó este enfoque arquitectónico para garantizar que la lógica del motor sea completamente agnóstica de la historia. Esto permite cumplir estrictamente con el requisito de modificabilidad: ejecutar una historia completamente nueva cambiando únicamente la ruta del archivo JSON de entrada, sin necesidad de alterar, recompilar o modificar una sola línea del código fuente.

5. Implementación y Pruebas (Unidad 7)

5.1 Descripción de Componentes

El desarrollo se estructuró en Python puro utilizando únicamente bibliotecas estándar del lenguaje (json, sys, unittest), garantizando portabilidad sin dependencias externas:

- motor.py: Núcleo lógico del sistema. Contiene el gestor de historias, la evaluación de condiciones booleanas, la aplicación de efectos sobre el estado de las variables en memoria, el control del historial de retroceso y el validador preventivo que intercepta errores estructurales antes de iniciar la partida.
- cli.py: Interfaz de usuario por consola. Se encarga de capturar la entrada del teclado, desplegar las opciones habilitadas y gestionar el flujo interactivo, incluyendo la prevención de bloqueos (*softlocks*) y mensajes de error controlados.
- test_motor.py: Suite automatizada de pruebas unitarias y de integración que verifica la robustez lógica del motor ante diversos escenarios de ejecución.

5.2. Resultados de Pruebas Automatizadas

La ejecución de la suite de pruebas automatizadas (unittest) cubre satisfactoriamente los escenarios requeridos por la rúbrica:

- U1 a U4 (Carga, Filtrado, Efectos y Validación): Comprueban con éxito la inicialización desde JSON, el filtrado correcto de opciones según el estado booleano, la aplicación múltiple de efectos y la detección estricta de errores de contenido (IDs duplicados, destinos inexistentes).
- U5 e I1 (Reinicio e Integración): Verifican que la selección de un final (final: true) o el reinicio de la partida limpien el estado adecuadamente y restauren los valores originales.
- U6 (Gestión del Cambio - Deshacer): Confirma que la acción de deshacer restaura con exactitud la escena y las variables previas, consumiendo la posibilidad de retroceso hasta realizar una nueva jugada válida, tal como lo exige el anexo de la hora 6.

6. Gestión del Cambio y Trazabilidad (Segunda Fase - Hora 6)

6.1 Análisis del Impacto del Cambio

• Modificación del Requisito: Se revierte el supuesto inicial de flujo unidireccional y se incorpora el requisito formal de retroceso (RF-07 / U6), permitiendo al lector deshacer su última decisión y revertir todas sus consecuencias.

• Impacto Arquitectónico y Reglas de Negocio Obligatorias (motor.py): Para cumplir con la solicitud del cliente, el motor guarda una instantánea (snapshot) de la escena y las variables previas, asegurando las siguientes 4 reglas:

1. Restauración y Un Solo Nivel: Deshacer recupera exactamente la escena y las variables anteriores a la última elección válida. Después de usarlo, la posibilidad de retroceso se consume y queda deshabilitada hasta realizar una nueva elección válida.
2. Inicio y Reinicio: Al iniciar la partida no hay retroceso disponible. La opción de reiniciar limpia el historial y restaura el estado inicial.
3. Entrada Inválida: Seleccionar una opción inexistente o no habilitada genera un mensaje de error, pero preserva intacto el historial de la última elección válida.
4. Desde un Final: Es posible ejecutar la acción de deshacer desde escenas finales (E4, E5 o E6), retornando a E3 y restaurando los valores de variables previos.

6.2. Actualización de Pruebas Automatizadas

La suite de pruebas (test_motor.py) se amplía con los siguientes escenarios obligatorios:

- Prueba U6 (Deshacer y Consumo): Verifica que la acción de deshacer restaura con exactitud la escena y las variables anteriores, consumiendo la posibilidad de retroceso hasta realizar una nueva jugada.
- Extensión de U5 (Reinicio y Limpieza): Comprueba que reiniciar la partida elimina por completo el historial de retroceso y restaura el estado inicial.
- Extensión de I1 (Deshacer desde un Final): Valida que es posible hacer retroceso incluso encontrándose en escenas terminales (E4, E5 o E6), regresando de forma segura a E3 y recuperando los valores previos.
- Caso de prueba de referencia (E2 a E3): Desde E2, la opción "Investigar" conduce a E3 marcando conoce_origen = true. Ejecutar la acción de deshacer retorna a E2 con conoce_origen = false. Un segundo intento consecutivo de deshacer no altera el estado.

6.3. Trazabilidad del Cambio

- Estado Anterior: El motor operaba estrictamente bajo un flujo hacia adelante sin opción de retorno.
- Estado Actualizado: El motor gestiona un historial de un solo nivel con control de reverso, validado tanto en ejecución normal como en escenas finales, manteniendo intacta la arquitectura modular por capas.

7. Conclusiones y Declaración de Aportes

7.1. Conclusiones

El desarrollo de "La última transmisión" permitió aplicar con éxito los principios de la ingeniería de software moderna bajo un marco ágil y estructurado por capas. La arquitectura orientada a datos (*Data-Driven*) logró un desacoplamiento efectivo entre la lógica de negocio y los contenidos en formato JSON, facilitando la modificabilidad del sistema. Asimismo, la incorporación del validador preventivo y la gestión controlada del retroceso (*undo*) garantizaron la robustez ante errores de contenido y una adaptación fluida a los requerimientos de la hora 6.

7.2. Declaración de Aportes

- Rodrigo Facio Lince: Diseño e implementación del núcleo lógico del motor (motor.py), desarrollo del validador preventivo, lógica de retroceso y estructuración de la suite de pruebas automatizadas (test_motor.py).

- Luis Gustavo Ruiz Teran: Desarrollo de la interfaz de consola (cli.py), creación de las historias en formato JSON, configuración del flujo de trabajo en Git/GitHub y redacción de la documentación técnica.